- Урок в прозе: https://proproprogs.ru/ml/ml-chto-takoe-mashinnoe-obuchenie-obuchayushchaya-vyborka-i-priznakovoe-prostranstvo (https://proproprogs.ru/ml/ml-chto-takoe-mashinnoe-obuchenie-obuchayushchaya-vyborka-i-priznakovoe-prostranstvo)
- Телеграм-канал: https://t.me/selfedu_sb (https://t.me/selfedu_sb)
- Телеграм-чат: https://t.me/machine_learning_selfedu (https://t.me/machine_learning_selfedu)

- К. В. Воронцов: https://ya-r.ru/2020/05/07/vorontsov-kurs-mashinnoe-obuchenie-2019-shkola-analiza-dannyh/ (https://ya-r.ru/2020/05/07/vorontsov-kurs-mashinnoe-obuchenie-2019-shkola-analiza-dannyh/)
- Лекции К. В. Воронцова: https://www.youtube.com/watch?v=SZkrxWhI5qM&list=PLJOzdkh8T5krxc4HsHbB8g8f0hu7973fK (https://www.youtube.com/watch?v=SZkrxWhI5qM&list=PLJOzdkh8T5krxc4HsHbB8g8f0hu7973fK)
- Лекции С. Николенко: https://www.youtube.com/watch?v=BPcicsBuRdc&list=PL-_cKNuVAYAWXoVzVEDCT-usTEBHUf4AF (https://www.youtube.com/watch?v=BPcicsBuRdc&list=PL-_cKNuVAYAWXoVzVEDCT-usTEBHUf4AF)
- Курс по Python: https://stepik.org/course/100707/ (https://stepik.org/course/100707/)
- Курс по Python ООП: https://stepik.org/course/116336/ (https://stepik.org/course/116336/)
- Курс по основам теории вероятностей: https://stepik.org/course/136024/ (https://stepik.org/course/136024/)
- Курс по NumPy: YouTube (https://www.youtube.com/playlist?list=PLA0M1Bcd0w8zmegfAUfFMiACPKfdW4ifD) | RuTube (https://rutube.ru/plst/535156)

Что такое машинное обучение. Обучающая выборка

Мы начинаем курс «Основы машинного обучения». Сразу отмечу, что это курс с математическим уклоном. Поэтому для его восприятия необходимо хорошо знать математическую базу:

- теорию вероятностей и математическую статистику;
- правила дифференцирования сложных функций и матриц;
- линейную алгебру.

Кроме того, на занятиях я буду использовать **язык Python** при реализации отдельных задач. Это необходимый минимум для начала изучения данного предмета. Далее, я буду полагать, что вы владеете этим материалом.

Конечно, машинное обучение – это целый мир и при разработке этого курса я активно пользовался сторонними источниками. Лучший охват этой темы в русскоязычном сегменте, на мой взгляд, дает профессор К. В. Воронцов. Ссылку на него вы найдете под этим видео. Для мен это было самое квалифицированное, понятное и полное изложение материала по данной теме с достаточно глубоким математическим уклоном и в то же время очень доступное. Я взял его кур лекций за основу своего курса. Кое что упростил, дополнил, в общем, это ни в коем случае не плагиат.

Помимо этого мне очень помогла книга С. Николенко «Глубокое погружение». Он рассматривает задачи машинного обучения с позиции теории вероятностей, что бывает очень полезно для понимания некоторых деталей. Также рекомендую его курс на ютуб (ссылка также под этим видео). Но он, скорее для тех, кто хорошо владеет математическим аппаратом теории вероятностей и математической статистики. Для новичков в этой области будет сложновато.

Конечно, я пользовался по мелочам и другими источниками. Сейчас все их уже и не вспомню. Плюс собственный скромный опыт по этой тематике. Но основные первоисточники я вам указа и советую их также посмотреть, так как там вы найдете очень много дополнительного материал В целом, я постарался сделать именно введение в машинное обучение для тех, кто мало касалс прежде этой темы и хотел бы разобраться в основах этого направления. А уже потом, думаю, будет гораздо понятнее и проще проходить курс профессора К. В. Воронцова и погружаться в другие полезные материалы.

Но, вернемся теперь непосредственно к нашей теме и давайте вначале разберемся, что скрывается за фразой **машинное обучение**. По простому – это алгоритмы, которые можно обучать, то есть, настраивать под решение конкретных задач. И здесь сразу возникают вопросы что это за задачи и как выполняется настройка алгоритмов? Начнем с первого. Далеко не все задачи следует решать методами машинного обучения. Например, если нам нужно найти точку минимума какой-либо функции, для простоты возьмем параболу:

$$f(x) = 2x^2 - 5x + 3$$

то ее легко решить классическими методами. Здесь достаточно вычислить производную по аргументу и приравнять ее к нулю:

$$\frac{df(x)}{dx} = 4x - 5 \Rightarrow x = \frac{5}{4}$$

Все, никаких алгоритмов машинного обучения не требуется. И везде, где можно относительно просто (с математической и вычислительной точки зрения) получить требуемый результат, не нужно применять никаких сложных, обучаемых алгоритмов.

А где без машинного обучения не обойтись или классическое решение задачи становится довольно сложным? Самый простой и очевидный пример – это классификация изображений. Например, нужно уметь различать кошек от собак, или идентифицировать человека по отпечатк пальца, или найти аномалии на рентгеновских медицинских снимках и т. п. Здесь везде исходны

данные (изображения) имеют сложную структуру и непонятно, как чисто математически описать универсальный алгоритм, который бы выдавал корректные результаты (в данном случае классификации). Поэтому идут на некоторую хитрость – создают алгоритм такой, чтобы он сам процессе обучения сформировал нужные критерии для решения задачи. В частности, это хорошо делают глубокие нейронные сети особенно в области распознавания образов на изображениях. Но об этом позже. Главное здесь понять, что не нужно «совать» алгоритмы машинного обучения первую попавшуюся задачу, возможно, она имеет гораздо более простое (в вычислительном плане) решение. И общее правило здесь такое: если ничего не помогает, то посмотрите в сторон обучаемых алгоритмов, которые могут дать приемлемое, но далеко не всегда точное решение.

Так какие же вообще существуют задачи машинного обучения? Обычно выделяют следующие категории:

- задачи классификации;
- задачи регрессии;
- задачи ранжирования.

Задача классификации – это когда требуется входные данные отнести к тому или иному классу (например, изображения разделить на кошек и собак, или распознать произнесенное слово, или по медицинским данным выдать диагноз и т.п.). В задачах регрессии обычно делают прогнозы виде вещественных чисел на основе входных данных (например, оценка курса валют по предыдущим показаниям, или прогноз объема продаж товара определенного вида, или просто построить прогноз некоторой функции по измеренным эмпирическим данным и т.д.). Наконец, третий тип задач подразумевает упорядочивание (по некоторому критерию) входного набора данных. Классический пример – это поисковые системы, которые ранжируют поисковую выборку по релевантности для каждого конкретного пользователя.

Все эти задачи, как правило, решаются с применением алгоритмов машинного обучения, т.к. входные данные имеют достаточно сложную структуру, а значит, нужно построить нетривиальные критерии для их обработки. Сделать это человеку часто просто не под силу, поэтому нужно подключать процесс обучения по так называемой обучающей выборке.

Обучающая выборка

И вот здесь мы подошли к первому важному моменту – что из себя представляет обучающая выборка? Изначально – это некие исходные данные, прямые результаты измерений. Это может быть то же самое изображение, звуковой сигнал, измерения роста, веса человека и т.п. Обычно все эти данные представляются набором некоторых чисел (все же мы имеем дело с вычислительной техникой, а она воспринимает только числа). Все эти измерения удобно представлять единым образом в виде векторов. Я буду их обозначать, следующим образом:

$$x_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{in} \end{bmatrix} = [x_{i1}, x_{i2}, \ldots, x_{in}]^T$$

(черточку сверху указывать не стану, т.к. по ходу изложения это будет и так понятно). Здесь представлено измерение некоего i-го объекта. Например, так можно представить i-е изображение, если все его пиксели вытянуть в одну строку – в один вектор, или амплитуды звукового сигнала, или пары измерений веса и роста. Я, думаю, понятно, что вектором можно представить любой набор измерений.

Далее, мы все эти наборы можем составить в матрицу:

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_l \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & \ldots & x_{1n} \\ x_{21} & x_{22} & \ldots & x_{2n} \\ \vdots & \vdots & & \vdots \\ x_{l1} & x_{l2} & \ldots & x_{ln} \end{bmatrix}$$

В данном случае матрица X составлена из l векторов и каждый вектор содержит n чисел. Кстати, в общем случае, размерность этих векторов может меняться, но обычно, в задачах машинного обучения их стараются делать равными. Так с ними проще работать.

Но матрица X это еще не обучающая выборка. Нам здесь для полноты картины не хватает выходных данных:

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = [y_1, y_2, \ldots, y_m]^T$$

Их еще называют **целевыми** (target). Что это за выходы? Например, в задачах классификации он могут принимать одно из следующих значений:

- $y \in \{-1, +1\}$ - при бинарной классификации;
- $y \in \{1, 2, \ldots, M\}$ - при M-классовой классификации (классы не пересекаются);
- $y \in \{0, 1\}^M$ - при M-классовой классификации с пересекающимися классами (здесь M – это размерность пространства).

В задачах регрессии каждый вход связан с одним или несколькими вещественными числами:

- $y \in \mathbb{R}$ - одномерная задача регрессии;
- $y \in \mathbb{R}^m$ - m-мерная задача регрессии.

В задачах ранжирования:

- Y – некоторое упорядоченное множество значений.

И вот вся эта совокупность данных $X^l = \{(x_i, y_i)_{i=1}^l\}$ уже может выступать в качестве обучающей выборки, так как каждому входному вектору (наблюдению) x_i поставлено в соответствие значение (или вектор) y_i. Такие данные еще называют **размеченными** и являются своего рода, «золотом», «валютой» для алгоритмов машинного обучения, т.к. их формирование, зачастую, - это самый сложный и длительный процесс, требующий участие одного или группы людей.

Но и это еще не все. Исходные результаты измерений $\{x_i\}$ могут иметь сложную структуру (например, при кодировании текстов, звуков или изображений) и трудно интерпретируемыми самими алгоритмами. В частности, до появления глубоких нейронных сетей (deep learning) в задачах классификации изображений инженеры формировали, так называемые, **вторичные признаки**: выделяли различные монотонные области, границы объектов с помощью различных фильтров. Доходило даже до того, что набрасывали на изображение случайные линии и пытались, затем, вычислять различные соотношения площадей. И такие подходы в разных задачах давали определенные результаты. По-английски это называется **feature extraction** и **feature engineering** – непростой и творческий процесс, создания более информативных признаков.

На уровне математики извлечение признаков из сырых исходных данных можно представить в виде функционального (в широком смысле этого слова) преобразования:

$$\{f_1(x), f_2(x), \ldots, f_k(x)\}$$

Здесь x – это вектор результатов исходных измерений, который затем, переводится в набор признаков, причем k - любое число, не обязательно равное n (исходному числу признаков – длина вектора x).

В результате, можно записать матрицу F, содержащую эти выделенные признаки:

$$F = \left\| f_j(x_i) \right\|_{l \times k} = \begin{pmatrix} f_1(x_1) & \ldots & f_k(x_1) \\ \vdots & & \vdots \\ f_1(x_l) & \ldots & f_k(x_l) \end{pmatrix}$$

Получаем новое признаковое пространство F, упрощающее работу алгоритмов машинного